

NutriFit

Guía de Defensa ante la Profesora

Borrador definitivo TFG · DAM 2025

Antonio Manuel Fresco Gómez · Preparación reunión presencial

Índice

1. ¿Qué es NutriFit? El pitch de 60 segundos
2. Arquitectura del sistema – la visión global
3. Flujo de login: explicación paso a paso
4. El Dashboard: corazón de la aplicación
5. Módulos de la aplicación: qué hace cada uno
6. Base de datos y Flyway: el esquema evolutivo
7. Stored Procedure: sp_resumen_diario
8. APIs externas integradas
9. Seguridad: los 5 vectores cubiertos
10. Tests: backend y frontend
11. Decisiones de diseño y por qué
12. Preguntas esperadas con respuestas preparadas
13. Guía para la demo en vivo
14. Lo que NO decir (errores a evitar)

1. ¿Qué es NutriFit? El pitch de 60 segundos

Aprende esto de memoria. Será la primera pregunta.

"NutriFit es una aplicación web fullstack para el seguimiento nutricional y de actividad física. El usuario registra sus comidas, ejercicios y peso corporal; la app calcula automáticamente sus necesidades calóricas según su perfil, le muestra su balance energético diario y le da una evaluación personalizada mediante inteligencia artificial. Incluye gamificación con retos y un NutriScore diario para motivar hábitos saludables."

Tecnologías en una frase

Frontend: React 19 con TypeScript, desplegado en Vercel.

Backend: Spring Boot 3.5 con Java 21, JDBC puro sobre PostgreSQL, desplegado en Render.

IA: OpenRouter con modelo Gemma (fallback DeepSeek).

APIs externas: OpenFoodFacts para alimentos, Wger para ejercicios.

Cifras del proyecto

Métrica	Valor
Archivos Java (backend)	157 archivos
Líneas de código Java	~10.900
Líneas TypeScript/React	~6.200
Módulos funcionales	16 módulos
Migraciones de BD (Flyway)	22 migraciones
Tests backend	114 tests
Tests frontend	17 tests
Páginas de la app	13 páginas

2. Arquitectura del sistema

NutriFit sigue una arquitectura **cliente-servidor REST** con separación total entre frontend y backend.



Capas del backend (patrón en 3 capas)

Cada módulo sigue la misma estructura:



Por qué esta arquitectura:

Separa responsabilidades. El controlador sólo entiende de HTTP. El servicio sólo de lógica de negocio. El repositorio sólo de SQL. Si mañana cambio PostgreSQL por MySQL, sólo toco los repositorios.

Despliegue

Componente	Plataforma	URL
Frontend React	Vercel	nutri-fit-snowy.vercel.app
Backend Spring Boot	Render	nutrifit-backend-ndoj.onrender.com

Base de datos PostgreSQL

Render (managed)

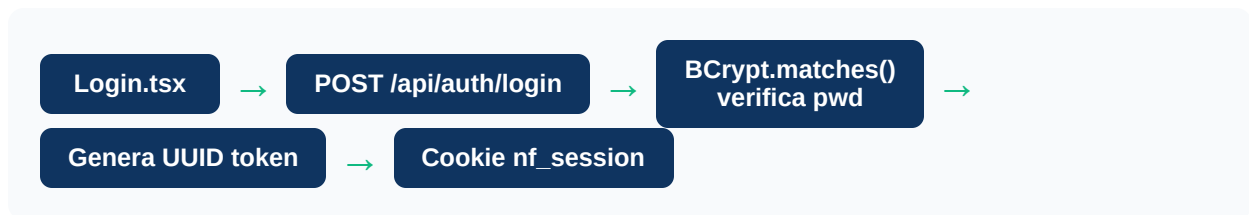
Interna al backend

3. Flujo de login: explicación paso a paso

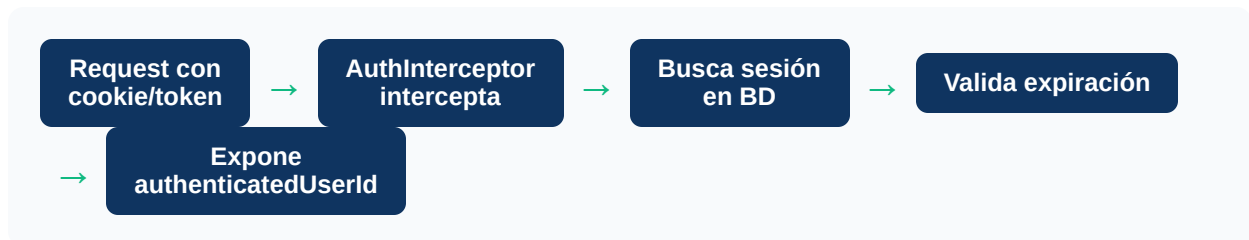
Registro de un nuevo usuario



Login



Cada petición autenticada



Punto clave: authenticatedUserId

El controlador nunca confía en el ID que manda el cliente. Usa el ID que extrae el interceptor de la sesión. Esto evita que el usuario A acceda a datos del usuario B (vulnerabilidad IDOR).

¿Por qué UUID y no JWT clásico?

El token UUID se almacena en base de datos. Esto permite invalidarlo al hacer logout de forma inmediata. Con JWT stateless, una vez emitido no se puede revocar hasta que expire.

Rate limiting en login

La clase `LoginRateLimiter` implementa una ventana deslizante: máximo 10 intentos por IP en 60 segundos. Si se supera, devuelve `429 Too Many Requests`. Protege contra ataques de fuerza bruta.

4. El Dashboard: corazón de la aplicación

Es la pantalla principal que el usuario ve al hacer login. Agrega datos de varios módulos en una sola vista.

Qué muestra el Dashboard

Elemento	De dónde viene	Qué calcula
Kcal consumidas	Stored Procedure + ResumenDiarioService	Suma de kcal de todas las comidas del día
Kcal quemadas	Registro de ejercicios	$MET \times \text{peso} \times \text{duración en horas}$
TDEE	PerfilServiceImpl	$TMB \text{ (Mifflin-St Jeor)} \times \text{factor actividad}$
Balance real	ResumenDiarioService	$Kcal \text{ consumidas} - TDEE - Kcal \text{ quemadas}$
NutriScore	GamificacionService	Puntuación 0-100 según 4 criterios
Racha de registro	GamificacionService	Días consecutivos con comidas registradas
Evaluación IA	EvaluacionIaService → OpenRouter	Análisis textual del día completo
Ventana de recuperación	RegistroEjercicioService	Si hubo ejercicio intenso, sugiere proteína+carbos

Fórmula TDEE (explícala si te preguntan)

Mifflin-St Jeor:

$TMB \text{ hombre} = 10 \times \text{peso(kg)} + 6.25 \times \text{altura(cm)} - 5 \times \text{edad} + 5$
 $TMB \text{ mujer} = 10 \times \text{peso(kg)} + 6.25 \times \text{altura(cm)} - 5 \times \text{edad} - 161$

$TDEE = TMB \times \text{factor de actividad}$
(Sedentario: 1.2 / Ligero: 1.375 / Moderado: 1.55 / Activo: 1.725 / Muy activo: 1.9)

NutriScore: cómo se calcula

El `GamificacionService` evalúa 4 criterios binarios (cumple o no cumple):

1. **Proteína:** ¿Ha consumido $\geq 0.8g \times \text{peso corporal en kg}$?
2. **Balance:** ¿El balance calórico está dentro de un rango razonable?
3. **Ejercicio:** ¿Ha registrado actividad física ese día?

4. **Variedad:** ¿Ha consumido ≥ 3 alimentos distintos?

La puntuación (0-100) determina el grado: A (≥ 80), B (≥ 65), C (≥ 50), D (≥ 35), E (< 35).

MacroRing

El componente `MacroRing.tsx` muestra un anillo circular con proteínas, grasas e hidratos de carbono. Usa Recharts (librería de gráficos para React). El anillo exterior representa el TDEE del usuario como referencia visual.

5. Módulos de la aplicación

Comidas

Registro de comidas diarias con alimentos y gramos. Cada comida puede tener múltiples alimentos. Permite navegar por fecha.

Alimentos

Base de datos de alimentos con valores nutricionales. Búsqueda por nombre. Permite añadir alimentos personalizados.

Escáner

Usa la cámara del dispositivo para leer códigos de barras. Consulta OpenFoodFacts y añade el producto automáticamente a la base de datos local.

Ejercicios

Catálogo de ejercicios importado de la API Wger. El usuario registra ejercicios con duración e intensidad. El sistema calcula kcal quemadas por MET.

Hidratación

Registro diario de vasos de agua consumidos. Muestra progreso hacia el objetivo diario (2L por defecto).

Peso Historial

El usuario registra su peso corporal. La app calcula la tendencia y estima la fecha para alcanzar el peso objetivo basándose en el balance calórico medio.

Plan Semanal

Planificador de comidas semanal (lunes-domingo). El usuario asigna comidas a días y tipos (desayuno, almuerzo, cena, snack).

Lista de Compra

Lista de productos para comprar. Permite marcar ítems como comprados. Puede sugerir alimentos basados en el plan semanal.

Retos

Sistema de desafíos predefinidos (ej: "30 días sin azúcar", "Beber 2L al día durante 7 días"). El usuario los acepta y hace seguimiento de su progreso.

Tendencias

Gráficas históricas de peso, NutriScore, macronutrientes semanales y actividad física. Datos de los últimos 30/90 días.

Perfil

Datos biométricos del usuario: peso, altura, edad, sexo, nivel de actividad, peso objetivo. Con estos datos se calcula el TDEE personalizado.

Opciones IA

El usuario puede configurar qué modelo de IA prefiere (Gemma o DeepSeek) para su evaluación diaria personalizada.

6. Base de datos y Flyway

¿Qué es Flyway?

Flyway es una herramienta de control de versiones para la base de datos. Cada cambio en el esquema se escribe como un archivo SQL numerado (V1, V2, V3...). Al arrancar la aplicación, Flyway comprueba qué migraciones ya se aplicaron y ejecuta solo las nuevas. Esto garantiza que la BD siempre esté sincronizada con el código.

Las 22 migraciones

Versión	Qué hace
V1	Tablas usuarios y alimentos base
V2	Tablas centrales del sistema
V3	Tabla de sesiones de autenticación
V4	Datos iniciales de alimentos (seed)
V5	Tablas para registro de comidas
V6	Tablas de ejercicios
V7	Datos iniciales de ejercicios (seed)
V8	Stored Procedure sp_resumen_diario
V9	Actualización del SP de resumen
V10	Índices para optimización de queries
V11	Datos expandidos de alimentos
V12	Tabla historial de peso
V13	Índices y restricciones adicionales
V14	Tabla de registro de hidratación
V15	Tablas de plan semanal
V16	Tablas de retos y desafíos
V17	Tabla de lista de compra
V18	Clasificación de tipos de ejercicio
V19	Catálogo completo de ejercicios
V20	Registro de intensidad anaeróbica

V21	Ajuste de restricciones de duración
V22	Tabla de configuración de IA por usuario

Tablas principales

Tabla	Propósito
<code>usuarios</code>	Datos de usuario: email, password hash, nombre
<code>sesiones</code>	Tokens activos con fecha de expiración
<code>perfiles</code>	Datos biométricos: peso, altura, edad, sexo, actividad
<code>alimentos</code>	Catálogo nutricional: kcal, proteínas, grasas, carbos por 100g
<code>comidas</code>	Comida registrada: usuario, fecha, tipo
<code>comida_alimentos</code>	Relación N:M entre comida y alimentos con gramos
<code>ejercicios</code>	Catálogo de ejercicios con MET
<code>registro_ejercicios</code>	Ejercicio realizado: usuario, fecha, duración, kcal quemadas
<code>peso_historial</code>	Registro histórico de pesajes
<code>agua_registros</code>	Ingesta diaria de agua
<code>retos</code>	Definición de retos disponibles
<code>usuario_retos</code>	Retos aceptados por usuario con progreso
<code>plan_semanal</code>	Planificación semanal de comidas
<code>lista_compra</code>	Items de la lista de compra
<code>usuario_ia_config</code>	Preferencias de modelo IA por usuario

7. Stored Procedure: sp_resumen_diario

Este es el punto más técnico que te puede preguntar la profesora. Explícalo bien.

¿Qué hace?

Calcula la ingesta nutricional total de un usuario para una fecha dada. Suma las calorías, proteínas, grasas e hidratos de carbono de todos los alimentos registrados en todas las comidas de ese día.

El SQL del Stored Procedure

Lógica de la query:

Hace un JOIN de tres tablas: `comidas` → `comida_alimentos` → `alimentos`

Para calcular las kcal de un alimento: $(kcal_por_100g \times gramos) / 100$

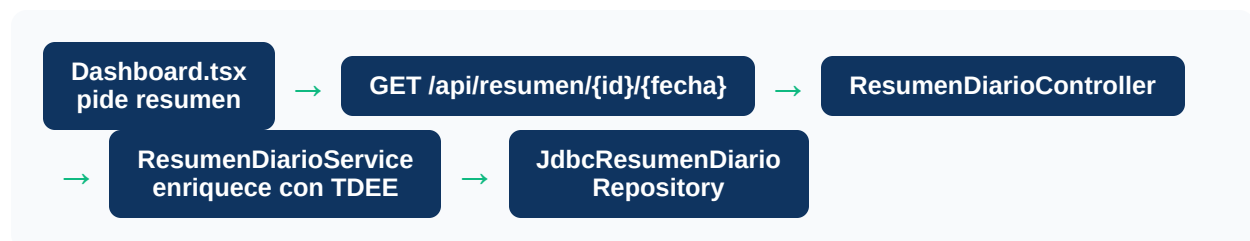
Agrupar por usuario y fecha, sumando todos los alimentos.

Usa COALESCE para devolver 0 si no hay datos (en vez de NULL).

¿Por qué existe además en Java?

La implementación principal está en `JdbcResumenDiarioRepository` (Java con JDBC). El stored procedure existe en la base de datos como alternativa demostrable y para cumplir el requisito del TFG de usar procedimientos almacenados.

¿Cómo lo llamas en la app?



8. APIs externas integradas

OpenFoodFacts (alimentos)

Qué hace:

API pública gratuita con información nutricional de productos de supermercado.

Cómo se usa:

El módulo Escáner lee el código de barras del producto, envía el código a OpenFoodFacts, recibe los valores nutricionales y los guarda en la base de datos local de alimentos.

Clase Java:

```
OpenFoodFactsService.java
```

Wger (ejercicios)

Qué hace:

API pública con un catálogo de ejercicios con información de músculos trabajados y valores MET.

Cómo se usa:

Al iniciar la app, importa el catálogo de ejercicios de Wger. El usuario puede buscar y registrar cualquier ejercicio.

Clase Java:

```
WgerService.java
```

OpenRouter (IA)

Qué hace:

Pasarela de acceso a múltiples modelos de lenguaje (LLMs) con una sola API.

Cómo se usa:

Cuando el usuario pulsa "Analizar día" en el Dashboard, el backend envía el contexto nutricional del día a OpenRouter.

Modelo principal:

Gemma (Google, gratuito).

Modelo de respaldo:

DeepSeek. Si Gemma falla, automáticamente intenta con DeepSeek.

Configuración:

Cada usuario puede elegir su modelo preferido en "Opciones IA".

Clase Java:

```
EvaluacionIaService.java
```

Qué se manda a la IA

El controlador `ResumenIaController` enriquece la petición con el contexto de los últimos 7 días antes de enviarlo al servicio IA:

- Kcal consumidas y quemadas del día actual
- Proteínas, grasas e hidratos del día
- TDEE personalizado del usuario
- Balance calórico real
- Media de kcal de los últimos 7 días
- Días con ejercicio en la última semana

9. Seguridad: los 5 vectores cubiertos

Ataque	Solución implementada	Clase/Fichero
SQL Injection	JdbcTemplate con <code>?</code> como placeholders. Nunca se concatenan valores del usuario en el SQL.	Todos los repositorios JDBC
IDOR (acceso a datos de otro usuario)	El interceptor extrae el userId de la sesión en BD. Los controladores usan ese ID, nunca el que manda el cliente.	AuthInterceptor + todos los Controllers
Contraseñas débiles / Rainbow tables	BCrypt con salt automático. Cada hash es único aunque la contraseña sea la misma.	PasswordService
Brute force en login	Máximo 10 intentos por IP en ventana deslizante de 60 segundos.	LoginRateLimiter
CORS / peticiones cruzadas	Whitelist explícita: solo Vercel y localhost. Ningún otro origen puede hacer peticiones a la API.	WebMvcConfig

Respuesta completa si te preguntan por seguridad:

"El proyecto implementa 5 capas de seguridad: SQL injection mediante prepared statements, IDOR con autenticación verificada en cada controlador, contraseñas con BCrypt que incluye salt automático, rate limiting contra fuerza bruta, y CORS con whitelist explícita."

10. Tests

Backend: 114 tests

Los tests de backend usan **JUnit 5** con **Testcontainers**. Testcontainers levanta un contenedor Docker real de PostgreSQL para cada ejecución de tests, lo que garantiza que los tests se ejecutan contra una BD real, no contra mocks.

¿Por qué Testcontainers y no mocks de BD?

Los mocks de base de datos pueden pasar el test aunque el SQL esté mal. Testcontainers ejecuta el SQL real contra PostgreSQL real, detectando errores que con mocks no se verían.

Qué se testea

- Repositorios JDBC: operaciones CRUD reales contra la BD
- Servicios: lógica de negocio (cálculos de TDEE, balance, NutriScore)
- Controladores: endpoints HTTP con MockMvc
- Seguridad: validación de tokens, rate limiting

Frontend: 17 tests

Los tests de frontend usan **Vitest** con **React Testing Library**. Testean componentes clave:

- `Login.test.tsx` : renderización y validación del formulario de login
- `Register.test.tsx` : registro de nuevos usuarios
- `AuthContext.test.tsx` : gestión del estado de autenticación
- `ProtectedRoute.test.tsx` : redirección a login si no autenticado
- `ErrorBoundary.test.tsx` : captura de errores de React

11. Decisiones de diseño y por qué

? ¿Por qué JDBC puro y no Hibernate / Spring Data JPA?

✓ JDBC da control total sobre las queries SQL. Con Hibernate, el ORM genera el SQL automáticamente y puede hacer consultas ineficientes. Con JDBC, cada query está escrita a mano y optimizada. También es más educativo: demuestra conocimiento real de SQL, no dependencia de abstracción.

? ¿Por qué React y no otro framework?

✓ React es el estándar de facto en frontend web. Su sistema de componentes reutilizables encaja perfectamente con una app modular como NutriFit. TypeScript añade tipado estático que reduce errores en tiempo de desarrollo.

? ¿Por qué Spring Boot?

✓ Spring Boot es el framework Java más usado en el mundo empresarial. Simplifica la configuración de la aplicación pero sin esconder la arquitectura. JDBC, validación, CORS e interceptores están todos configurados explícitamente, no por magia.

? ¿Por qué PostgreSQL?

✓ PostgreSQL es la BD relacional open-source más avanzada. Soporta funciones PL/pgSQL para los stored procedures, tiene excelente soporte de tipos de datos, y Render ofrece PostgreSQL gestionado gratuitamente.

? ¿Por qué tokens UUID y no JWT clásico?

✓ El token UUID se almacena en la tabla `sesiones` de la base de datos. Esto permite invalidar la sesión al hacer logout de forma inmediata. Con JWT stateless, una vez firmado el token, no se puede revocar hasta que expire.

? ¿Por qué OpenRouter para la IA?

✓ OpenRouter es una pasarela que da acceso a múltiples modelos de IA con una sola API. Permite cambiar de modelo sin reescribir código, y tiene modelos gratuitos como Gemma de Google. El fallback automático a DeepSeek garantiza disponibilidad del servicio.

? ¿Por qué Flyway para la BD?

✓ Flyway mantiene el esquema de BD bajo control de versiones, igual que git mantiene el código. Si el proyecto se despliega en un servidor nuevo, Flyway aplica automáticamente todas las migraciones en orden. Nunca hay que ejecutar scripts SQL a mano.

12. Preguntas esperadas con respuestas preparadas

? Explícame qué hace tu proyecto

✓ NutriFit es una aplicación fullstack de nutrición y fitness. El usuario registra sus comidas diarias, ejercicios y peso. La app calcula automáticamente el balance calórico según su perfil biométrico y le da una evaluación personalizada con IA. Tiene gamificación con retos, NutriScore diario y análisis de tendencias históricas.

? ¿Cómo funciona el login?

✓ El usuario manda email y contraseña. El backend verifica la contraseña con BCrypt. Si es correcta, genera un token UUID aleatorio, lo guarda en la tabla sesiones con fecha de expiración, y lo devuelve como cookie. En cada petición posterior, el AuthInterceptor valida ese token contra la BD.

? ¿Cómo calculas las calorías?

✓ Los alimentos tienen kcal por 100 gramos. Cuando el usuario registra "150g de pollo", calculamos $(\text{kcal_por_100g} \times 150) / 100$. El stored procedure `sp_resumen_diario` suma esto para todos los alimentos de todas las comidas del día.

? ¿Qué es el TDEE?

✓ Total Daily Energy Expenditure: las calorías que el cuerpo necesita al día. Lo calculamos con la fórmula Mifflin-St Jeor: $\text{TMB} = 10 \times \text{peso} + 6.25 \times \text{altura} - 5 \times \text{edad}$ (± 5 según sexo). Luego multiplicamos por el factor de actividad del usuario. Es la referencia para saber si está en déficit o superávit calórico.

? ¿Qué es un Stored Procedure y para qué lo usas?

✓ Un stored procedure es una función SQL que vive dentro de la base de datos. El mío, `sp_resumen_diario`, recibe el ID de usuario y una fecha, y devuelve el total de calorías, proteínas, grasas e hidratos de ese día. La ventaja es que la lógica de agregación se ejecuta en la BD, más cerca de los datos.

? ¿Cómo protege tu app contra ataques?

✓ Cinco capas: prepared statements contra SQL injection, verificación de sesión en cada endpoint contra IDOR, BCrypt para contraseñas, rate limiting de 10 intentos/minuto contra fuerza bruta, y CORS con whitelist para bloquear peticiones desde orígenes no autorizados.

? ¿Por qué JDBC y no un ORM como Hibernate?

✓ JDBC me da control total sobre el SQL. Con un ORM, el framework genera las queries automáticamente y puede hacer joins o subconsultas ineficientes sin que yo lo vea. Escribir el SQL a mano es más trabajo pero más eficiente y educativo: demuestra que sé SQL, no solo que sé usar una librería.

? ¿Qué hace Flyway?

✓ Flyway gestiona las migraciones de base de datos. Cada cambio en el esquema es un archivo SQL numerado: V1, V2... Al arrancar la app, Flyway comprueba cuáles se han ejecutado ya y aplica solo las nuevas. Es como git pero para la base de datos: el esquema tiene historial y versiones.

? ¿Cómo funciona la IA?

✓ Cuando el usuario pulsa "Analizar día", el backend recoge los datos nutricionales del día más el histórico de 7 días y los manda a OpenRouter, que es una pasarela de IA. El modelo Gemma (de Google) genera un texto de evaluación personalizado. Si Gemma falla, automáticamente intenta con DeepSeek como respaldo.

? ¿Cómo funciona el escáner?

✓ El componente usa la librería html5-qrcode para acceder a la cámara del dispositivo y leer el código de barras. Manda ese código al backend, que consulta la API pública de OpenFoodFacts. Si encuentra el producto, devuelve sus valores nutricionales y los guarda en la base de datos local para uso futuro.

Si te lo preguntan en demo: entra a la página Escáner y escanea un producto real. Si no tienes a mano, explica el flujo mostrando el código.

? ¿Por qué React y no Angular o Vue?

✓ React es la librería más demandada en el mercado laboral. Su enfoque basado en componentes y hooks es más flexible que Angular (más orientado a empresas grandes) y tiene un ecosistema enorme. Para una SPA como NutriFit encaja perfectamente.

? ¿Tienes tests? ¿Qué testeas?

✓ Sí, 114 tests en el backend y 17 en el frontend. En el backend uso JUnit con Testcontainers: levanta un PostgreSQL real en Docker para cada test, sin mocks. Esto garantiza que el SQL funciona de verdad. En el frontend uso Vitest con React Testing Library para testear componentes clave: login, registro, rutas protegidas.

13. Guía para la demo en vivo

ANTES de la reunión

Asegúrate de tener datos reales en tu cuenta. Si la BD de producción está vacía, registra comidas, ejercicios y pesajes previamente para que la demo tenga datos que mostrar.

Orden recomendado de la demo

1. **Login:** Muestra que solo entras con credenciales válidas.
2. **Dashboard:** El punto fuerte. Muestra kcal, TDEE, balance, NutriScore, racha. Explica brevemente qué es cada dato.
3. **Evaluación IA:** Pulsa "Analizar día" y espera la respuesta. Explica que llama a OpenRouter con el contexto del día.
4. **Comidas:** Añade una comida con alimentos. Muestra cómo se calculan las kcal al añadir gramos.
5. **Escáner:** Si tienes un producto a mano, escanéalo. Si no, explica el flujo.
6. **Tendencias:** Muestra las gráficas históricas (peso, NutriScore, macros).
7. **Perfil:** Muestra los datos biométricos y explica que de aquí sale el TDEE.
8. **Retos:** Muestra el sistema de gamificación.

Riesgo en demo: la IA puede tardar 10-15 segundos

Render duerme los servicios gratuitos tras 15 min de inactividad. El primer request después de ese tiempo tarda 30-60 segundos en "despertar" el servidor. Entra a la app 5 minutos antes de la reunión para que el backend esté activo.

Tip: abre DevTools antes de la demo

Si la profesora pregunta cómo se comunica el frontend con el backend, puedes abrir la pestaña Network del navegador y mostrar las peticiones HTTP en tiempo real.

14. Lo que NO decir (errores a evitar)

No digas: "Lo hice con IA"

Aunque hayas usado herramientas de asistencia, tú tomaste cada decisión técnica, revisaste cada línea, entiendes qué hace cada parte. El proyecto es tuyo.

No digas: "No sé qué hace eso exactamente"

Si señala algo que no recuerdas al momento, no lo admitas con esas palabras. Di: "Eso está en el módulo X, que se encarga de Y. Déjame mostrártelo." Y navega al código o a la pantalla correspondiente.

No digas: "Es que Spring Boot lo hace automáticamente"

Nunca te escondas detrás del framework. Siempre explica QUÉ hace el framework y POR QUÉ lo usas. "Spring Boot gestiona el ciclo de vida del servidor HTTP, la inyección de dependencias y la configuración automática, lo que me permite centrarme en la lógica de negocio."

Cuidado con el vocabulario técnico

Si usas un término, asegúrate de poder explicarlo. No digas "ORM" si no puedes definir qué es. No digas "JWT" si estás usando UUID tokens (que no son JWT). No digas "microservicios" si tienes un monolito (que es lo correcto para un TFG).

No des respuestas de una palabra

Si pregunta "¿qué es Flyway?", no respondas solo "una librería de migraciones". Añade para qué sirve en tu proyecto y qué problema resuelve.

Mentalidad correcta para la reunión:

No estás siendo examinado sobre teoría. Estás explicando un proyecto que tú construiste. Habla como el autor que eres. Si no sabes algo al momento, dilo con confianza: "Eso lo tendría que revisar en el código, pero la idea general es..."

Resumen rápido – Lo que tienes que dominar sí o sí

Concepto	Respuesta de una frase
¿Qué es NutriFit?	App fullstack de nutrición: registro de comidas, ejercicios, IA y gamificación.
¿Qué tecnologías?	React + TypeScript (frontend), Spring Boot + Java 21 + JDBC + PostgreSQL (backend).
¿Cómo funciona el login?	BCrypt verifica contraseña → token UUID en BD → cookie → AuthInterceptor valida.
¿Qué es el TDEE?	Calorías diarias necesarias. Fórmula Mifflin-St Jeor × factor de actividad.
¿Por qué JDBC?	Control total del SQL, sin magia de ORM, más eficiente y educativo.
¿Qué es Flyway?	Control de versiones para el esquema de la BD. 22 migraciones secuenciales.
¿Stored procedure?	sp_resumen_diario: suma kcal/macros de todas las comidas del día.
¿Cómo protege contra SQL injection?	JdbcTemplate con prepared statements, nunca concatenación de strings.
¿Qué hace la IA?	OpenRouter llama a Gemma (fallback DeepSeek) con contexto del día para evaluación.
¿Cuántos tests?	114 backend (Testcontainers + PostgreSQL real), 17 frontend (Vitest).
¿Dónde está desplegado?	Frontend en Vercel, backend + BD en Render.